TeamFlow 🚀

Technical Presentation, Architecture Walkthrough & Live Demo Guide

Rewamp Digital Internship Selection

Stack: Next.js 15 • Express • Prisma • PostgreSQL

⏱️ 5–7 Minute Showcase Agenda

Time	Segment	Key Focus & Takeaway
0:00 – 1:00	**Pitch & Architecture**	Monorepo layout, clean tech stack, 100% real database aggregates, zero mock data.
1:00 – 2:30	**Visual Bug Reporting (Differentiator)**	Screen recording, voice memos, clipboard paste (Ctrl+V), error log viewer, HTTP 206 streaming.
2:30 – 4:00	**Core Agile Workflows & Collaboration**	Tasks vs Bugs separation, Kanban boards, Resolution Modal, in-app notification bell & audio chime.
4:00 – 5:00	**Security & Engineering Architecture**	Magic-byte file sniffing, scoped 404 security, reporter-only permissions, signed streaming tokens.
5:00 – 6:00	**Verification, Testing & Q&A**	35 passing integration tests, strict TypeScript (0 errors), live transactional email delivery.

🖥️ Live Demo Walkthrough (Step-by-Step Script)

Part 1: Introduction & Architecture Setup (0:00 – 1:00)

> **Say:** *"Good morning/afternoon! Today I'm showcasing* **TeamFlow**, *an agile team collaboration and visual bug reporting platform built with Next.js 15, TypeScript, Express.js, Prisma ORM, and PostgreSQL.*
>
> *I designed TeamFlow as a production-grade monorepo where* **every metric, task, recording, and notification is 100% real and backed by PostgreSQL database aggregates and file storage***—there are zero simulated counters or fake timers."*

Part 2: The Core Differentiator — Visual Bug Reporting (1:00 – 2:30)

ACTION Click the red **"Report a Bug"** button in the top navigation bar.

> **Say:** *"In traditional issue trackers, developers receive vague reports like 'the checkout button is broken'. In TeamFlow, we solved this with an integrated* **Visual Bug Reporting suite***:*
>
> - **Live Screen Recording:** *Click 'Record Screen', select the window, and record a live video with microphone audio.*
> - **Voice Memo Audio:** *Record audio voice notes with waveform pulsation and accessible timers.*
> - **Clipboard Paste (Ctrl+V):** *Paste screenshots straight from the OS clipboard into the ticket with automatic thumbnails.*
> - **Diagnostic Error Log Viewer:** *Paste stack traces into an escaped, monospaced viewer with one-click copy.*
>
> *Our upload engine uses a* **Two-Step Resilient Architecture***: the backend ticket is created first, then media files upload sequentially with real progress percentages. If an upload fails, users can retry individual files without duplicating tickets.*
>
> *When opened on the issue page, videos and audio stream via* **HTTP 206 Partial Content Range requests**, *allowing teammates to seek and scrub immediately without waiting for the full file to download."*

Part 3: Workflows, Notifications & Resolution Documentation (2:30 – 4:00)

ACTION Navigate to **Tasks**, open the Kanban board, and move an item to **DONE**.

> **Say:** *"Next, let's look at TeamFlow's task management and collaboration workflows:*
> - **Dedicated Workflows:** *We cleanly separate* **Tasks** *(assigned work items) from* **Issues/Bugs** *(defect tracking). Bugs follow a* **Collaborative Bug Solving** *model—open to all project members to investigate—while Tasks are assigned to specific team members.*
> - **Strict Media Security:** *To preserve audit integrity, media attachments on bug reports can strictly only be added or deleted by the original reporter.*
> - **Interactive Resolution Notes:** *When moving an issue to 'DONE', TeamFlow prompts for root cause and resolution documentation, automatically appending a formatted note to the discussion thread so team knowledge is preserved.*
> - **Real-Time Notification Bell:** *Click the bell icon. Whenever a task is assigned, updated, or commented on, teammates receive an in-app alert with an unread badge and an audio chime, alongside transactional emails."*

Part 4: Role-Based Metrics & Scoped Dashboards (4:00 – 5:00)

ACTION Switch between Admin view and Member view.

> **Say:** *"Notice our dashboard design:*
> - *When an* **Admin** *logs in, they see organization-wide operational health: active projects, unassigned tasks, and overdue items.*
> - *When a* **Member** *logs in, their dashboard counters ('My Open Tasks', 'My Overdue Tasks') are strictly scoped to their personal workload—preventing other members' tasks from cluttering their view."*

💡 Engineering Implementation & Architectural Highlights

1. Pluggable Storage Driver Architecture

Clean driver abstraction (`StorageDriver`) decoupling business logic from file storage:

- **LocalDiskDriver (Dev):** Zero-credential local development with custom HTTP 206 byte-range chunk streaming.
- **S3Driver (Production):** Cloudflare R2 / AWS S3 object storage compatibility.

2. Magic-Byte Security & Scoped 404s

Multi-layered backend security enforcement:

- **Raw Magic-Byte Sniffing:** Reads raw buffers using `file-type` to block executable files disguised with fake extensions and block SVG XSS vectors.
- **Scoped 404s:** Cross-organization access attempts return 404 instead of 403, preventing resource enumeration leaks.

3. Performance & Defensive State

Optimized for sub-100ms UI responsiveness:

- **Client Preview Cache:** Instant 0ms issue rendering while background revalidation finishes.
- **Decoupled Dependencies:** React hooks use stable refs to completely prevent infinite re-render cycles.

4. Real Email Delivery & Notifications

Complete communication pipeline:

- **In-App Bell & Web Audio Chime:** Real-time notifications for task assignments, self-assignments, and comments.
- **Live SMTP / Resend:** Automated transactional email dispatch for invites, security logins, and task assignments.

✅ **Quality Assurance Summary:** 35 automated integration tests running via Vitest with 100% pass rate. TypeScript type check passes with 0 errors (`npx tsc --noEmit`).

🎯 Anticipated Interview Questions & Winning Answers

Q1: Why did you choose PostgreSQL and Prisma instead of MongoDB?

Answer: "Project management is inherently relational. Tasks belong to Projects, Projects belong to Organizations, and Issues have Repositories, Assignees, and Comment threads. PostgreSQL guarantees referential integrity, foreign key cascades, and ACID transactions. Prisma provides end-to-end type safety, auto-generated migrations, and prevents SQL injection by default."

Q2: How does your media streaming work locally without AWS S3?

Answer: "I implemented an HTTP 206 Partial Content Range streaming engine in `LocalDiskDriver`. When a browser requests bytes 0-1048576, the server parses the `Range` header, creates a readable fs stream with start/end offsets, and streams raw chunks with `Content-Range` headers. This allows users to scrub through 5-minute screen recordings immediately without waiting for a 50MB file to finish downloading."

Q3: How do you handle role permissions between Admins, PMs, and Members?

Answer: "RBAC is enforced centrally in backend middleware using organization memberships. Admins have global org management; Project Managers can create projects and assign tasks; and Members are restricted to visual bug reporting, commenting, and updating their assigned tasks. Even if someone tampers with client-side UI, the backend rejects unauthorized actions with 403 or scoped 404s."

Q4: What would you improve or add if you had another week?

Answer: "I would add WebSockets/Socket.io for collaborative multi-user live cursors on the Kanban board, and integration with GitHub/GitLab to automatically close TeamFlow issues when commit messages reference ticket keys like `TF-1`."

📂 Source Code Mapping: "How and From Where You Achieved This"

When asked to walk through the exact codebase and implementation files, refer directly to this architecture map:

Feature / Capability	Source Code Location	Underlying APIs & Implementation Mechanism
Live Screen Recording	`client/src/hooks/useScreenRecorder.ts`	Browser `navigator.mediaDevices.getDisplayMedia` + Web Audio API `AudioContext` to mix system audio and mic commentary into a `MediaRecorder` stream (`video/webm`).
Voice Memo Recording	`client/src/hooks/useAudioRecorder.ts`	`navigator.mediaDevices.getUserMedia({ audio: true })` + `MediaRecorder` with real-time waveform pulse timer and accessible `aria-live` status indicators.
Two-Step Resilient Upload	`client/src/app/bug-reports/new/page.tsx`	Step 1 creates the PostgreSQL ticket (`POST /api/issues`); Step 2 uploads media sequentially via native `XMLHttpRequest.upload.onprogress` for real upload % with per-file retry.
HTTP 206 Streaming Engine	`server/src/lib/storage/local-disk.driver.ts`	Parses `Range: bytes=start-end` headers, creates `fs.createReadStream(path, { start, end })` , and returns status 206 with `Content-Range` for instant seeking/scrubbing.
Raw Magic-Byte Sniffing	`server/src/modules/attachments/attachments.service.ts`	Reads raw Uint8Array buffer signatures via `file-type` library; refuses to trust file extensions or client MIME types to block disguised executables and SVG XSS attacks.
In-App Notification Bell & Chime	`client/src/components/ui/NotificationBell.tsx` `server/src/modules/notifications/`	Stores alerts in PostgreSQL `notifications` table. Client polls every 30s; synthesizes a pleasant dual-tone chime (D5 587Hz → A5 880Hz) via native Web Audio oscillators without external MP3s.
Role-Based Access & Scoped 404s	`server/src/middleware/policies.ts` `server/src/modules/issues/issues.service.ts`	Enforces `Role.ADMIN` , `Role.PROJECT_MANAGER` , and `Role.MEMBER` . Cross-organization access returns 404 instead of 403 to prevent resource enumeration attacks.
Resolution Documentation Modal	`client/src/components/issues/IssueDetailView.tsx` `server/src/modules/issues/issues.service.ts`	Prompts for root cause and resolution notes when moving status to `DONE` . Saves to `Issue.resolutionNotes` and auto-posts an audit comment to the discussion thread.
Role-Scoped Dashboard Metrics	`server/src/modules/dashboard/dashboard.service.ts` `client/src/app/dashboard/page.tsx`	Live database aggregates (`prisma.issue.count`). Scopes Member metrics strictly to `assigneeId: userId` while Admins see workspace-wide totals.
Relational Database Schema	`server/prisma/schema.prisma`	Full relational schema with `User` , `Organization` , `Project` , `Issue` , `Comment` , `Attachment` , `ActivityLog` , and `Notification` .

🔑 Quick Demo Reference

Role	Email	Password	Port / URL
ADMIN	`admin@teamflow.dev`	`Password123!`	`http://localhost:3000`
PROJECT_MANAGER	`pm@teamflow.dev`	`Password123!`	`http://localhost:3000`
MEMBER	`member@teamflow.dev`	`Password123!`	`http://localhost:3000`